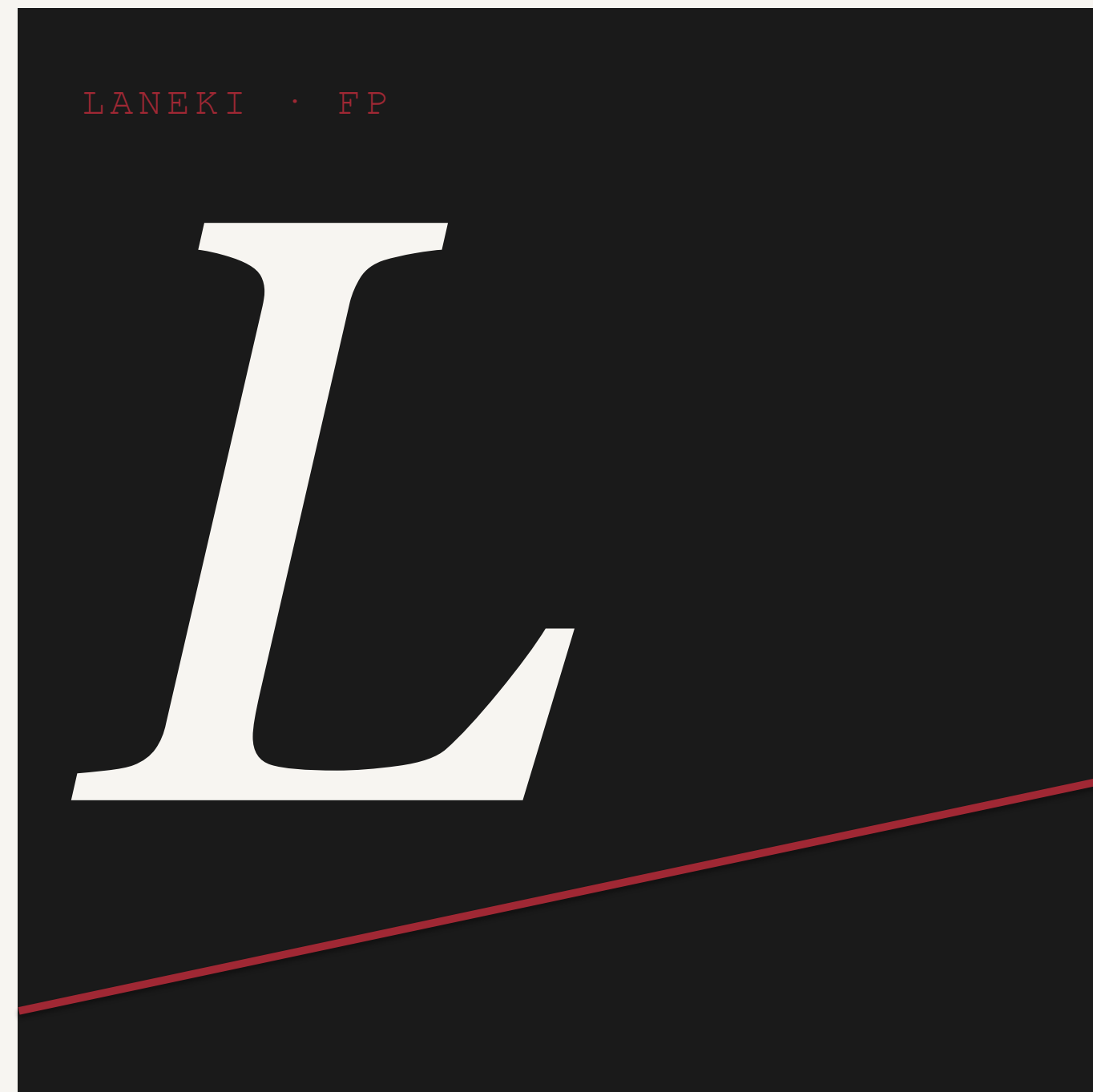
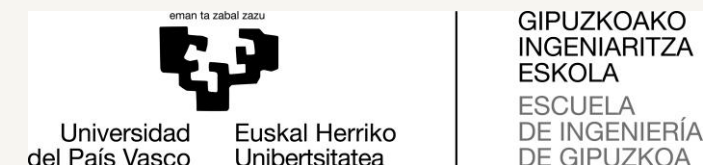


Datuen *Ingeniaritza*

Bizitza-zikloa, biltegiatzea, arkitekturak, irenstea eta datu formatuak

Zikloa Adimen Artifiziala eta Big Data

Modulua Big Data Sistemak



Datuen Ingeniaritza

01 Datuen bizitza-zikloa

02 Biltegiatzea

03 Big Data Arkitekturak

04 Datuen irenstea eta ETL

05 Datu formatuak

06 Faker datu sintetikoak

“

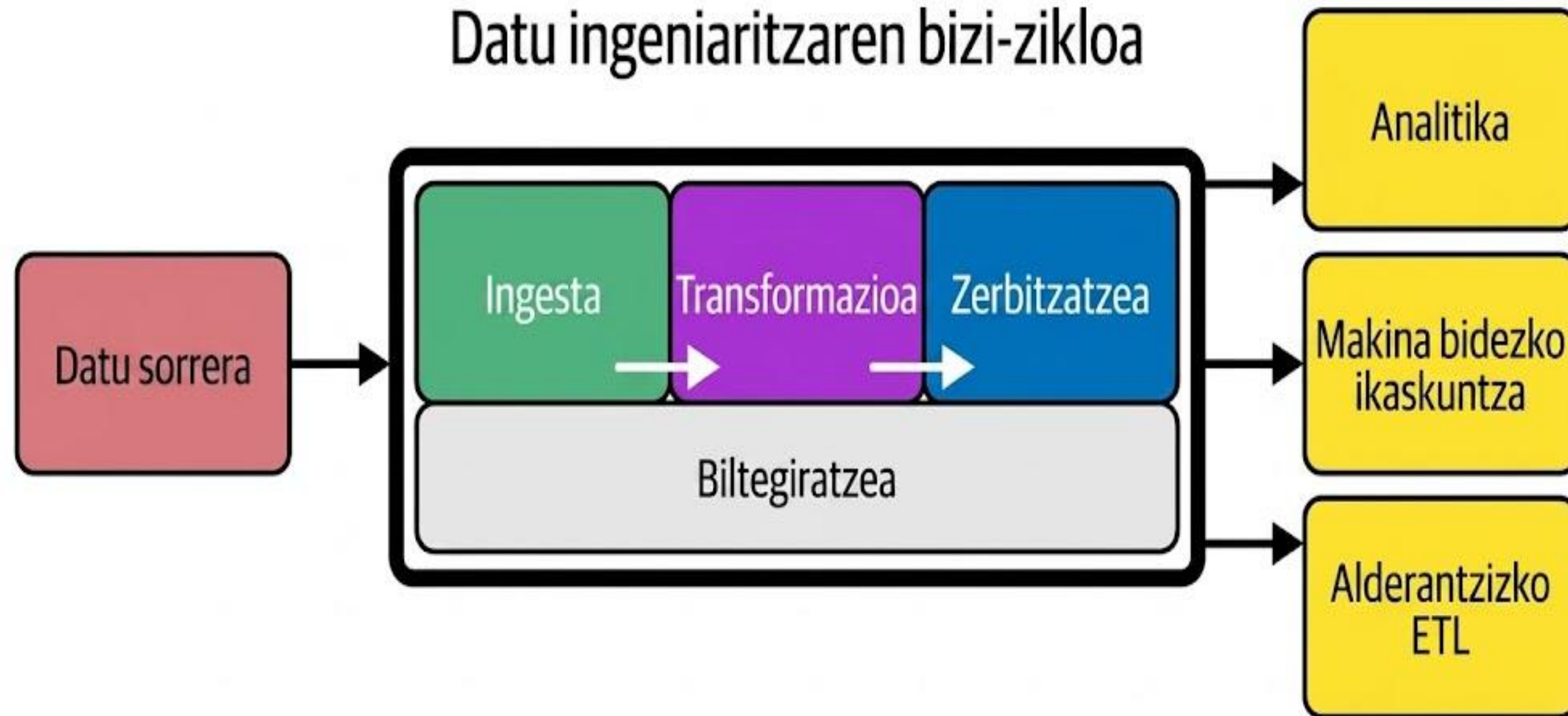
Datu gordinak berreskuratzen dituzten eta analitika eta machine learning bezalako erabilera-kasuetarako informazio koherente eta kalitate handiko datuak sortzen dituzten sistemen eta prozesuen garapena, inplementazioa eta mantentzea.

— FUNDAMENTALS OF DATA ENGINEERING · JOE REIS & MATT HOUSLEY

01

Datuen Bizitza-Zikloa

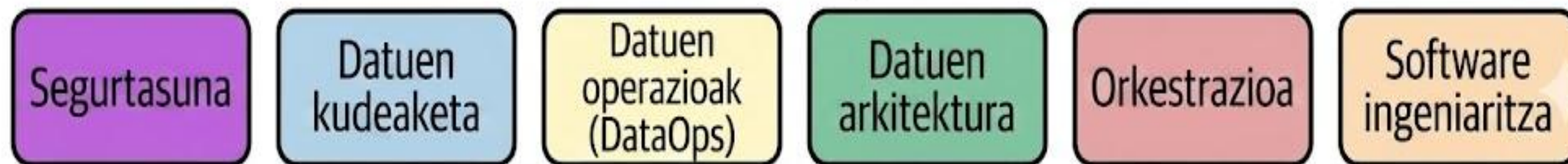
Datu ingeniartzaren bizi-zikloa



Datuen Bizitza-Zikloa

Bizitza-zikloa ez da lineala — iteratiboa da. Datuak sortu eta kontsumitu artean hainbat fase daude, guztiak arlo transbertsalek (segurtasuna, gobernantza) zeharkatzen dituztelarik.

Korronte-azpiak:



Sorrera → Biltegiatzea → [Irenstea, Eraldaketa, Zerbitzatzea] → Kontsumoa.

Bizitza-Zikloaren 3 Fase Nagusiak

01 Sorrera (Generation)

Nork, non, nola eta noiz sortzen diren datuak — analogikoak edo digitalak, fitxategiak, APIak, OLTP datubaseak, logak, IoT gailuak.

02 Biltegitratzea (Storage)

Datuak non eta nola gordetzen diren — HDD/SSD/RAM, datu-base erlazionalak, NoSQL, Data Lake/Warehouse/Lakehouse.

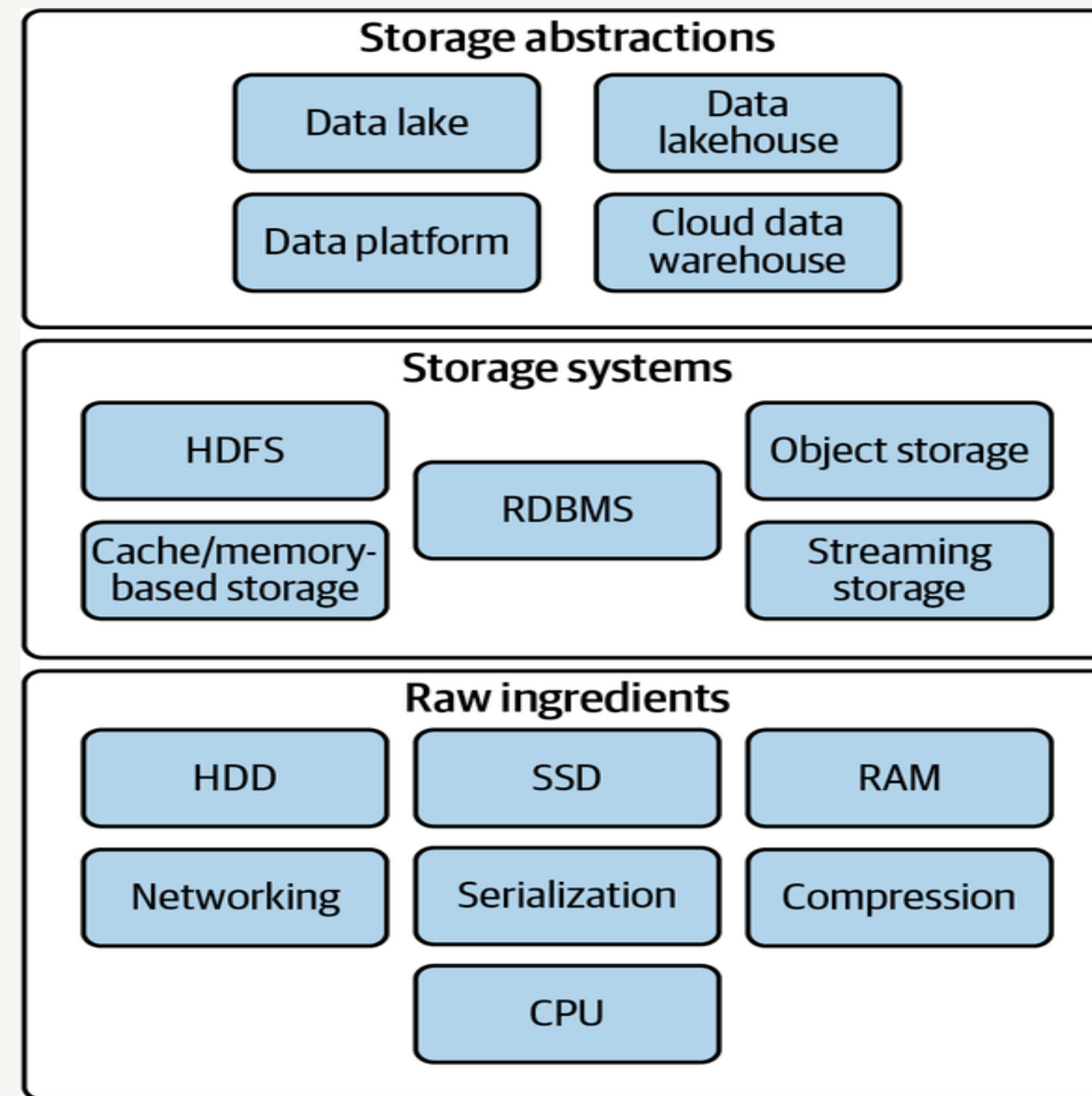
03 Kontsumoa (Serving)

Datuak nori eta nola entregatzen zaizkion — analitika, machine learning, dashboardak, aplikazioak.

02 Biltegiratzea

Biltegiratzeko Teknologia Fisikoak

- **HDD (disko magnetikoak)** — 200-300 MB/s · 500 IOPS ·
kostu baxua · artxibo hotzak
- **SSD (disko solidoak)** — hainbat GB/s · hamar milaka IOPS
· aktibo diren datuak
- **RAM** — ~100 GB/s · milioi bat IOPS · SSD baino 1000x
azkarragoa · cache
- **Cloud objektu-biltegiratzea** — S3, Blob Storage ·
eskalagarria · HDD antzeko kostua



Teknologia bakoitzak abiadura/kostu trade-off ezberdinak ditu.

Biltegiratzeko Abstrakzioak

SOLUZIOA	DATU MOTAK	GAITASUN NAGUSIA
Data Warehouse	Egituratuak soilik	OLAP kontsulta optimizatuak — AWS Redshift, Snowflake, BigQuery
Data Lake	Edozein mota — gordinak	Edozein formatu gordetzea — HDFS, AWS S3, Azure Blob Storage
Data Lakehouse	Egituratu + egitura gabe	Bien onena — Delta Lake (Databricks), Apache Iceberg, Snowflake
Cache/RAM	Sarri erabilitakoak	Latentzia minimoa — Redis, Memcached, aplikazio memorian

03

Big Data arkitekturak

Batch ala streaming?

Batch: hasiera eta amaiera duen datu-multzoa, multzoka (adib. gaueko karga). Bolumen handirako; latentzia ez kritikoa.

Streaming: datuak etengabe, iristen diren heinean (adib. iruzur-detekzioa). Latentzia kritikoa.

Pitfall: 'denbora erreala' $\neq$ 'berehalakoa' — tarte txiki batean erantzuten du.

Pitfall: 'streaming' askok barnean **micro-batch** erabiltzen dute.

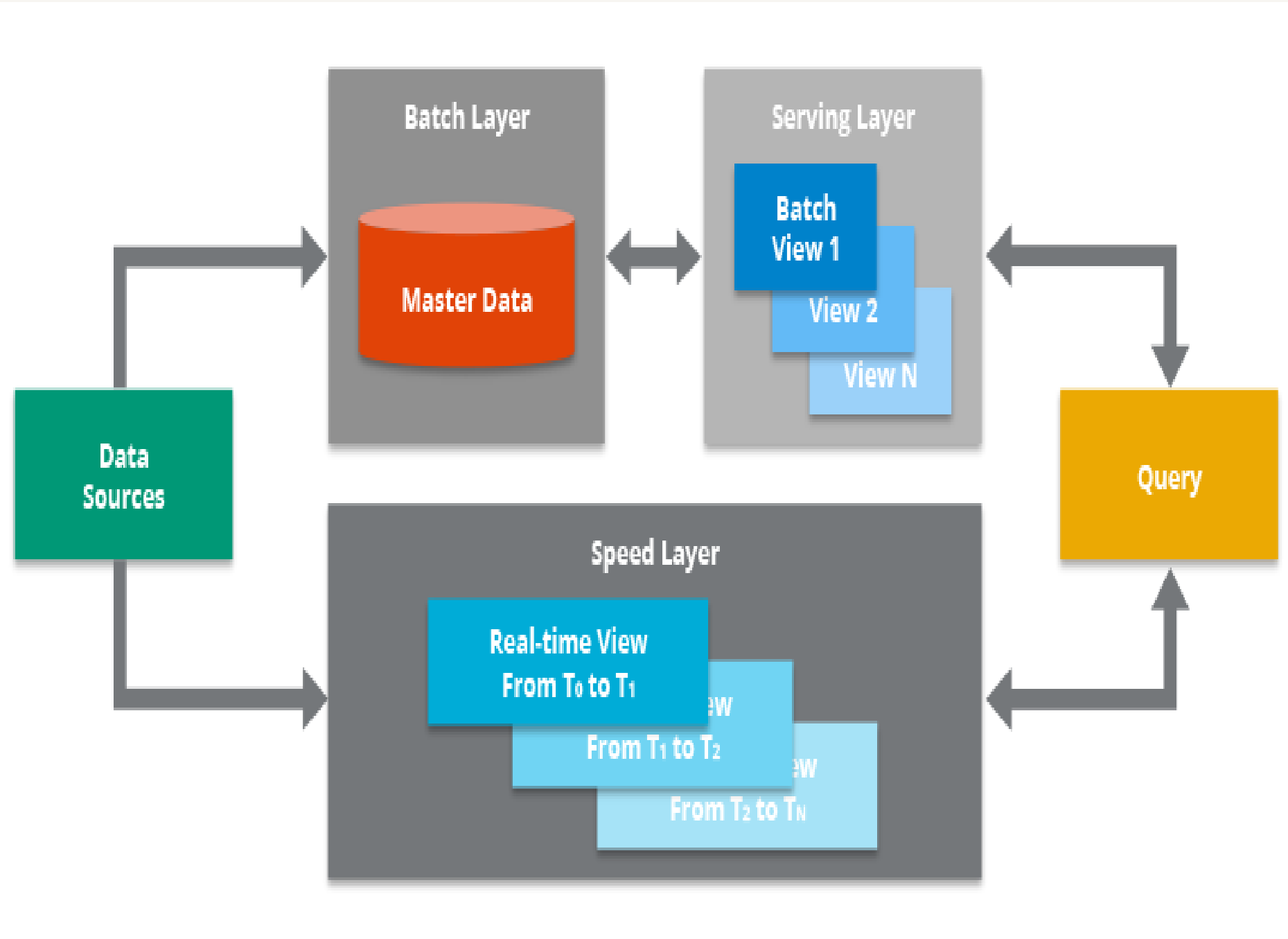
Batch vs Streaming Prozesatzea

ALDERDIA	BATCH	STREAMING
Definizioa	Datu-multzoak periodikoki	Etengabe, iritsitakoan
Latentzia	Minutuak / Orduak	Segundoak / Milisegundoak
Bolumena	Oso handiak — GB/TB	Gertaera txikiak, jarraian
Adibidea	Hileko nomina, gaueko txostena, ML entrenamendu	Ordainketa iruzurrak, trafiko alertak, IoT sentsores
Tresnak nagusiak	MapReduce, Spark jobs, Python scripts	Kafka, Spark Streaming, Apache Flink, Storm

— LAMBDA ARKITEKTURA

LAMBDA Hiru Geruzen Egitura

- **Batch Geruza** — datu historikoak, aldaezinak, emaitza zehatzak, latentzia altua
- **Speed/Streaming Geruza** — denbora errealekoak, inkrementalak, latentzia baxua
- **Serving Geruza** — bi geruzatako emaitzak konbinatu eta kontsultei erantzun



Irudi 2 Batch + Streaming + Serving geruzak batera.

Lambda Arkitektura · Fluxua Urratsez Urrats

01 Datu berria bikoiztuta bidali

Informazio berria Batch Geruza eta Speed Geruza bietara bidaltzen da aldi berean.

02 Batch Geruzak historikoa kudeatzen du

Datu gordinak aldaezin gordetzen ditu eta periodikoki Batch Views sortzen ditu Serving Geruzarako.

03 Speed Geruzak latentzia konpentsatzen du

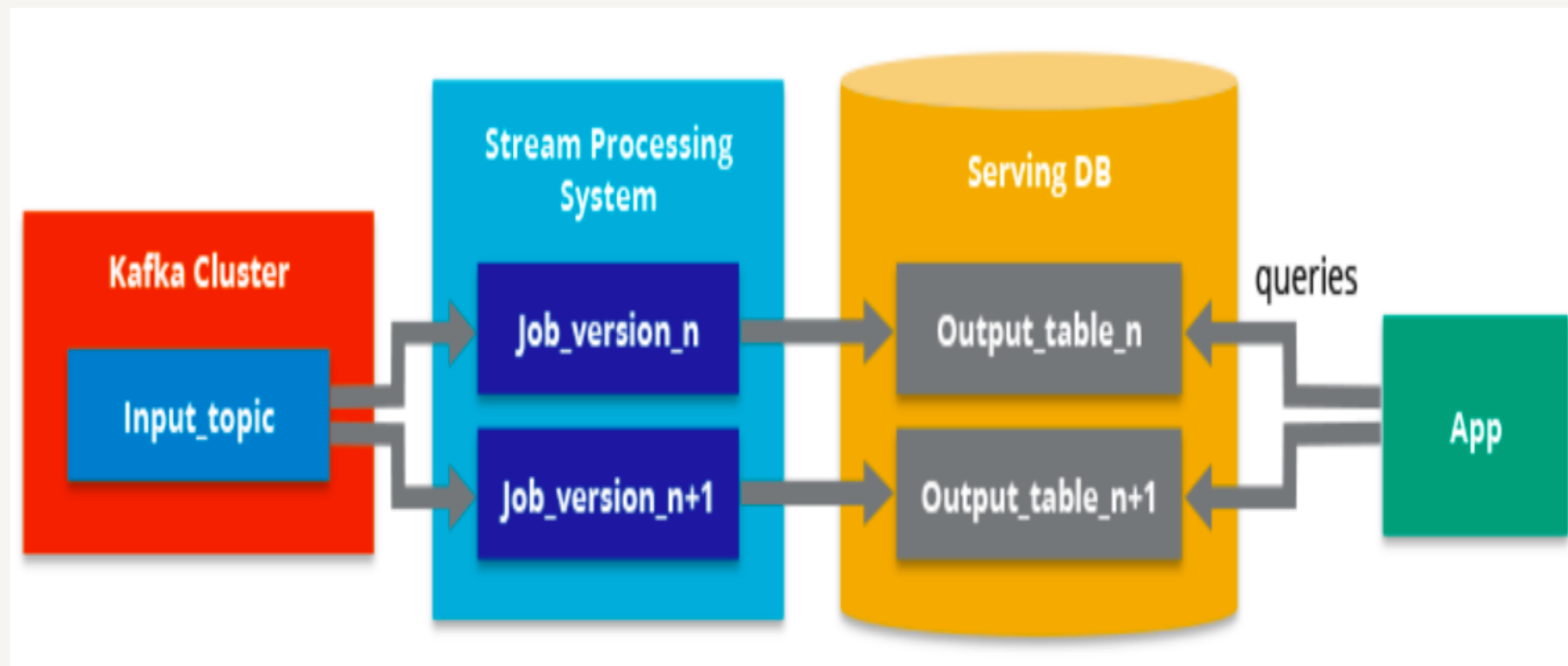
Batch-aren eguneraketa tardea betetzen du denbora errealeko ikuspegiak sortuz.

04 Serving Geruzak kontsultak erantzuten ditu

Batch Views eta Real-time Views konbinatuz kontsulten erantzunak eraikitzen dira.

KAPPA

Lau Zutabeak



Irudi 4 Streaming soilik — batch geruza desagertu.

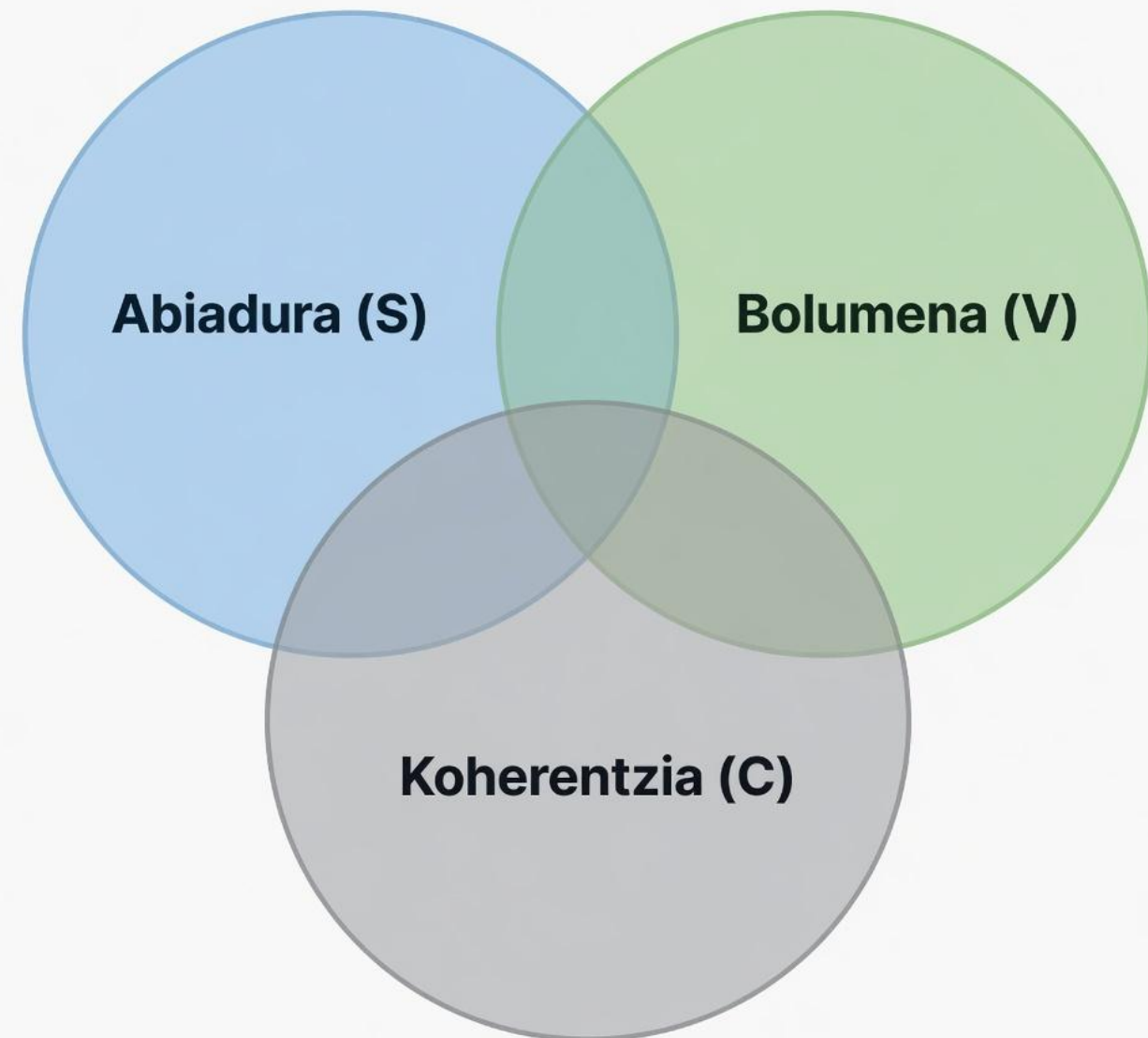
- **Dena stream da** — batch eragiketak streaming-aren azpimultzo dira
- **Datu aldaezinak** — eraldatu gabe gordetzen dira jatorrizkoak
- **Prozesatze-fluxu bakarra** — kode eta mantentzea nabarmen murriztu
- **Berrabiarazte posiblea** — prozesatzea aldatu eta konfigurazioa aldatu emaitzak aldatzeko

Lambda vs Kappa · Konparaketa

ALDERDIA	LAMBDA	KAPPA
Prozesatze-bideak	Bi bide: batch + streaming	Bide bakarra: streaming
Konplexutasuna	Altua — bi sistema mantendu	Baxuagoa — sistema bakarra
Zehaztasuna	Altua (batch emaitzak)	Streaming kalitatearen araberakoa
Datu historikoak	Ondo — batch geruza dago	Berrabiarazte beharrezkoa
Erabilera egokia	ML entrenamendu, analisi osoa batch-ekin	Batch eta streaming algoritmo berdinak direnean

— SCV PRINTZPIOA

SCV printzipioa



- **Speed (S)** — datua jaso eta emaitza sortu arteko denbora
- **Consistency (C)** — emaitzaren zehaztasuna (guztiak vs lagina)
- **Volume (V)** — prozesatu daitekeen datu-kopurua
- **3-tik 2 bakarrik**

SCV · Hiru Trade-off Posibleak

S + C (Gabe V): Zehaztasun altuko emaitzak azkar lortu daitezke, baina datu-bolumen handiak ezin dira prozesatu. Erabilera-kasua: datu-base txiki baten kontsulta azkarra.

C + V (Gabe S): Datu-bolumen handia modu koherentean prozesatu, baina erantzun-denbora luzea. Erabilera-kasua: hileko salmenta-txostena osoa (batch).

V + S (Gabe C): Datu-bolumen handiak azkar prozesatu, baina lagin-teknikak behar dira — emaitza ez da guztiz koherentea. Erabilera-kasua: denbora errealeko sare sozialeko analisia.

Lambda-ren erantzuna: Bi geruza konbinatzeak S+C+V hurbiltzea ahalbidetzen du praktikan — ez perfektua, baina nahikoa.

04

Datuen irenstea eta ETL

Zer da ETL eta zertarako?

Arazoa: datuak iturri askotan daude, formatu eta kalitate desberdinekin — ezin dira zuzenean aztertu.

ETL: **E**xtract (atera) → **T**ransform (garbitu/eraldatu) → **L**oad (helmugan kargatu).

Pipeline: datuak iturritik helmugara eramaten dituen prozesua. ETL pipeline **mota jakin bat** da.

Printzipioa: diseinua negozio-arazotik abiatu, ez teknologiatik.

Datu-Pipeline-a



Iturri batetik informazioa hartu,
garbitu/eraldatu, eta helmuga berri batean
idatzi. Prozesua **iteratiboa** da.

Fase eta teknologia ezberdinez osatuta dagoen prozesu bat da Datu Pipelinea — datuen mugimendua edo prozesatzea definitzen duena.

Beharrezkoak dira datuak sortzen diren sistemetan datu hauek ezin direlako aztertu — **OLTP sistemak** (CRM, ERP) ez dira analitikako karga jasateko pentsatuak.

Prozesatze analitikoa konputazionalki garestia da — sistema bereizituetan egin behar da.

Pipeline-aren 4 Faseak

- **Ingesta** — Datuak iturrietatik bildu eta pipeline-ra ekarri
- **Biltegitratzea** — Datuak modu iraunkorrean gorde
- **Prozesatzea** — Datuak **batch** edo **streaming** bidez irakurri, garbitu, eraldatu eta kalkuluak egin
- **Analisia eta bistaratzea** — Prozesatutako datuak aztertu eta emaitzak txosten, grafiko edo dashboardetan erakutsi



Irudi 2 Ingesta → Biltegitratzea → Prozesatzea → Analisia eta Bistaratzea.

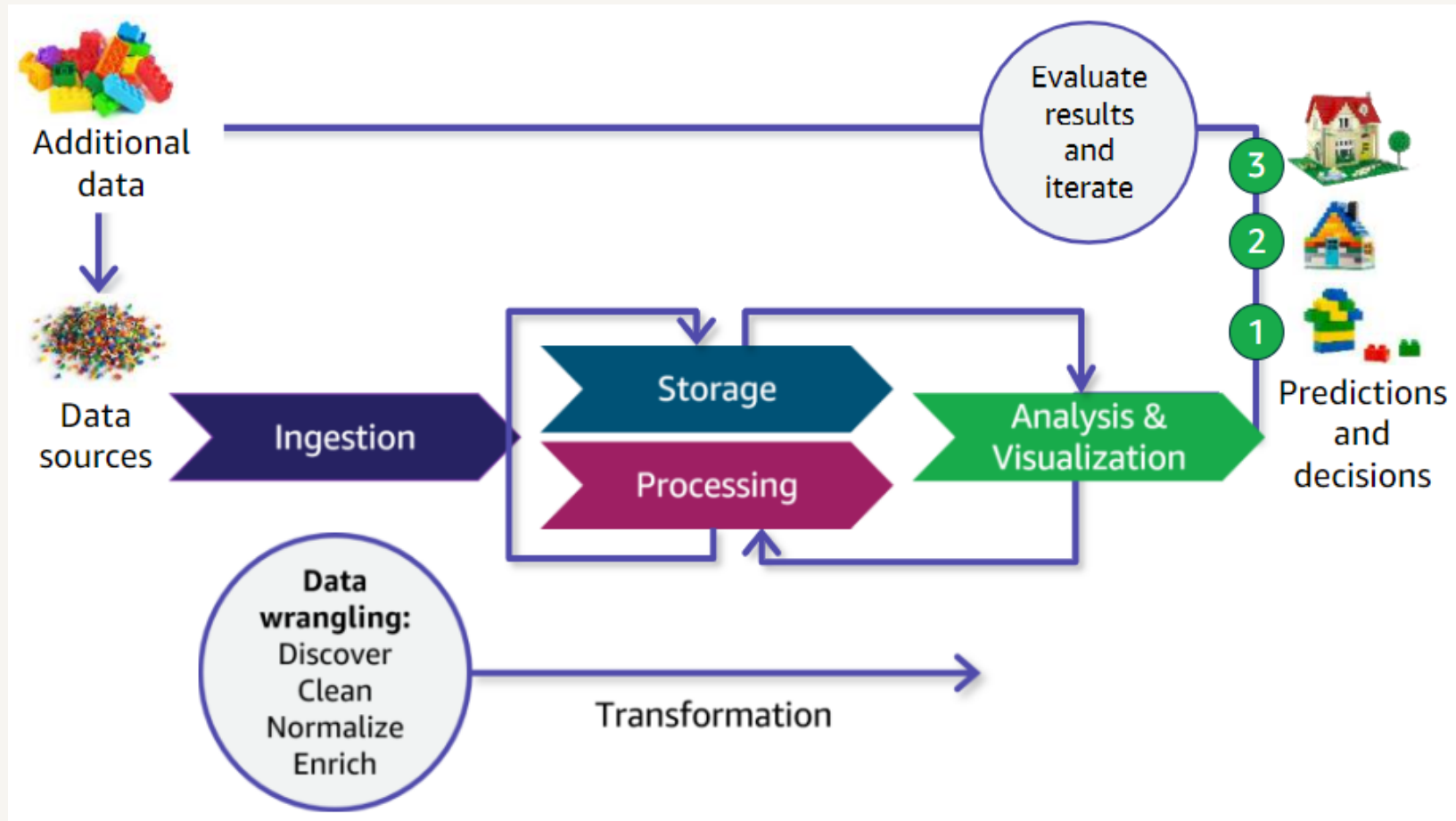
— DATU-PIPELINE · ITERAZIOA

Pipeline-a Prozesu Iteratiboa Da

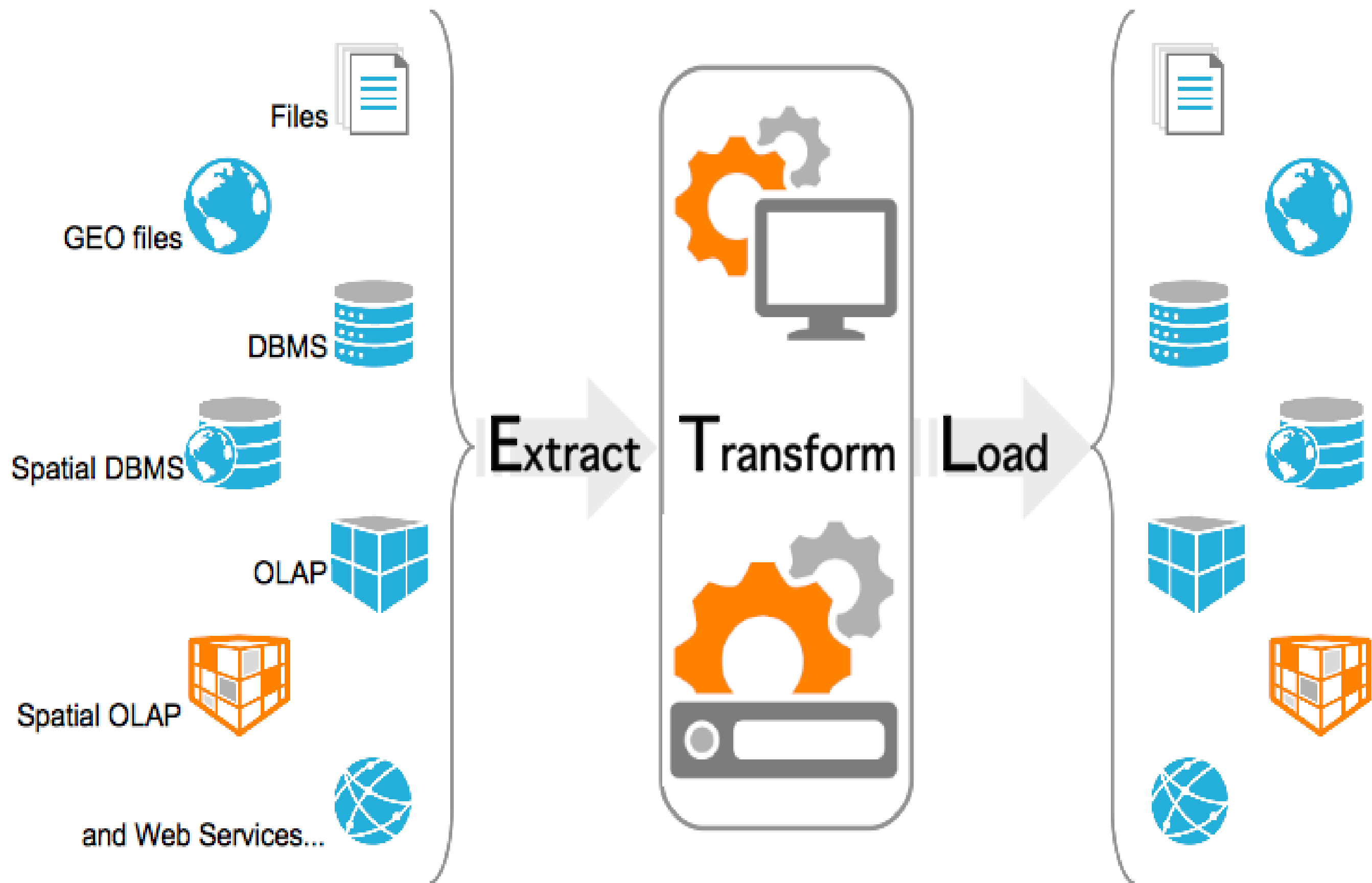
Prozesua **iteratiboa** da: hipotesia → datuak egiaztatu → falta bada, datu berriak bildu.

Analisi fasean espero den emaitza lortzen ez bada, **ingestara itzuli** datuak lortu edo aldatzeko.

Data Wrangling: datu gordinetik azken egoerara iristeko ekintza guztiak — garbiketa, normalizazioa, eraldaketa.



Irudi 3 Behar diren datuak falta badira, pipeline osotik igaro behar da berriro.



— ETL · FASEAK

ETL: Erauzketa, Eraldaketa eta Karga

Big Data testuinguruan ETL tresnek honako ezaugarriak behar dituzte:

1. formatu ezberdinekiko malgutasuna (JSON, CSV, Parquet)
2. Eskalagarritasuna
3. Akats-tolerantzia
4. Iturri eta helmuga anizkoitzen konektoreak.

Irudi 4 Iturri anizkoitzeko datuak leku bakarrean biltzeko aukera ematen du.

ETL · Hiru Faseetan Zehar

Extract Transform Load (ETL)

01 Erauzketa (Extract)

Jatorrizko sistemetatik datuak bildu — fitxategi lauak, datu-base erlazionalak, REST APIak, sare sozialak. Datuak egiazkoak direla egiaztatu. Jatorrizko sisteman inpaktua minimizatu behar da.

02 Eraldaketa (Transform)

Espero den formatua eta edukia lortzeko aldaketak egin — kodeketa-aldaketak, bikoiztuak ezabatu, iturri ezberrinak gurutzatu, adierazleak sortu. Ez du informazio garrantzitsua galdu behar.

03 Karga (Load)

Helmugako sistemara datuak gorde — Data Warehouse edo datu-base mota. Transakzio-denbora minimizatu. Indizeak, partizioak eta commit tamainak kontuan hartu.

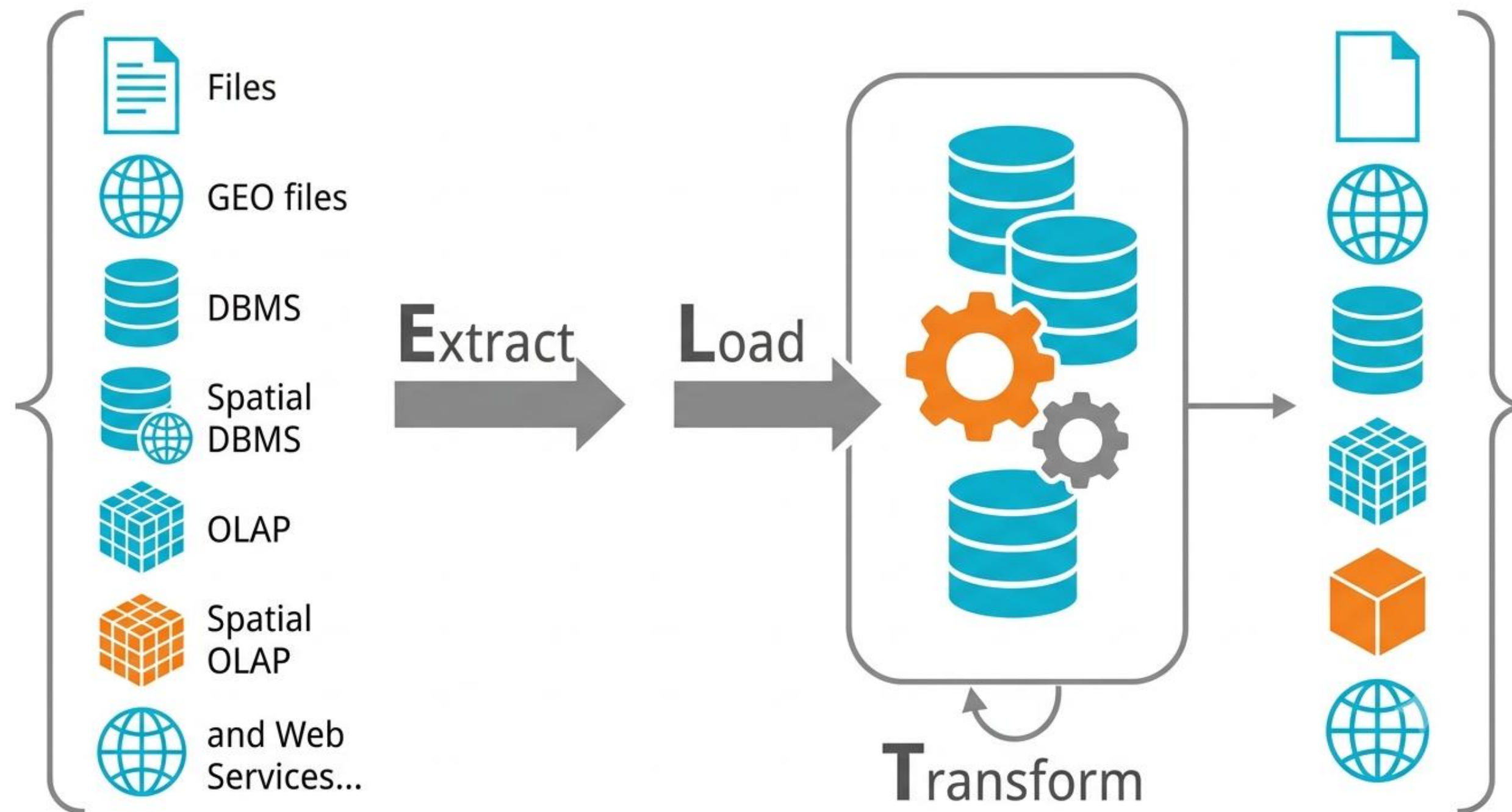
— ELT · FASEAK

ELT: Extract → Load → Transform

E – datuak bildu (jatorriari gutxien eragin)

L – gordina kargatu, ezer galdu gabe (data lake-an, dagoen-dagoenean).

T – helmugan bertan hobetu, ez asmatu informazio berririk (indizeak, partizioak, formatuak hor sortzen dira).



ELT · ETL-ren Bilakaera

Extract Load Transform (ELT)

ELT

Erauzketa, Karga eta Eraldaketa — siglak ordenan aldatzen ditu. Iturri anizkoitzeko datuak eraldatu gabe zuzenean Data Lake edo hodeiko biltegitratze-sistemara kargatzen dira. Handik, datuak negozio-helburu ezberdinen arabera eraldatzen dira.

ZERGATIK ELT NAGUSITU DA?

Cloud-eko biltegitratze merkea + prozesatze-potentzia handiak ELT-ri abantaila eman dio: datuak berehala eskuragarri daude eta analisten eskuetan uzten da eraldaketa — ez soilik ingeniarietan.

ETL vs ELT

	ETL (TRADIZIONALA)	ELT (MODERNOA)
Eraldaketa	Kargatu AURRETIK	Kargatu OSTEAN (lake-n)
Nork	Ingeniariek, zentralizatua	Analistek SQL/Python
Datuak noiz prest	Eraldaketa amaitzean	Berehala (gordin)
Big Data-n	Gutxiago	Gehiago — merkeagoa

ETL kode bidez: Pandas eta DuckDB

ETL prozesuak tresna grafikoekin edo kode bidez egin daitezke.

Python ingurunean erabiltzen diren bi tresna hauek dira:

■ PANDAS

Python-en datuak lantzeko liburutegia

- Datuak taula moduan kudeatzen ditu (DataFrame)
- CSV, Excel, JSON... fitxategiak irakur ditzake
- Datuak iragazi, eraldatu, elkartu eta gorde ditzake

```
df = pandas.read_csv("produktuak.csv")
```

■ DUCKDB

Datuak SQL bidez analizatzeko motor arina

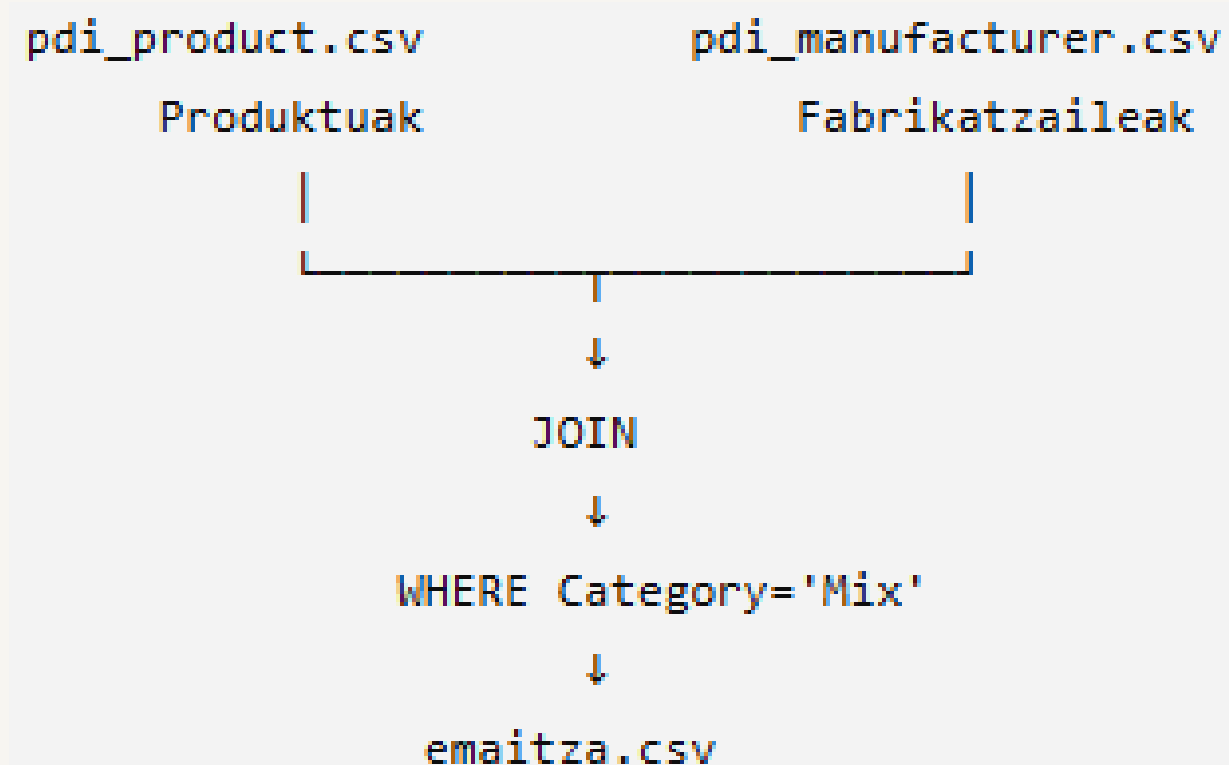
- Ez du zerbitzaririk behar
- CSV eta Parquet fitxategiak **zuzenean kontsulta** ditzake
- SELECT, WHERE, JOIN, GROUP BY... erabil ditzakegu

```
SELECT *  
FROM read_csv('produktuak.csv')  
WHERE Category = 'Mix'
```

ETL praktikan: DuckDB

HELBURUA: ETL prozesu bat kode errealean egitea, DuckDB eta SQL erabiliz.

Bi datu-iturri izango ditugu:



Zer egingo dugu?

E – EXTRACT (datuak irakurri):

```
read_csv('pdi_product.csv')  
read_csv('pdi_manufacturer.csv')
```

T – TRANSFORM (datuak eraldatu):

Bi fitxategiak elkartu eta Mix kategoriako produktuak bakarrik aukeratuko ditugu.

```
LEFT JOIN ...  
WHERE Category = 'Mix'
```

L – LOAD (emaitza gorde):

```
emaitza.csv
```

ETL praktikan: DuckDB

CSV → irakurri → elkartu eta iragazi → gorde

```
import duckdb

df = duckdb.sql("""
    SELECT p.*, f.Manufacturer
    FROM read_csv('pdi_product.csv') p
    LEFT JOIN read_csv('pdi_manufacturer.csv') f
        ON p.ManufacturerID = f.ManufacturerID
    WHERE p.Category = 'Mix'
""").to_df()

df.to_csv("emaitza.csv", index=False)
```

05

Datu formatuak

Bi Familia Nagusi: Testua eta Binariora

- **Testua (CSV, XML, JSON)** — gizakientzat irakurterrazak; normalean tamaina handiagoa eta prozesamendu-kostu handiagoa
- **Binariora (Avro, Parquet)** — trinkoagoak eta makina bidez prozesatzeko eraginkorragoak
- Big Data-n formatu bat aukeratzean: **bateragarritasuna, eskema tamaina, errendimendua, zatigarritasuna**
- **Formatuen egiturak prozesamendu paraleloan eragiten du** — Parquet bezalako formatuek Big Data analitika eraginkorra errazten dute

user_id	country	subscription_type
1	US	Free
2	UK	Paid
3	ES	Paid

Nola Antolatzen Dira Datuak Diskoan?

Row storage		Column storage	
Row 1	1	user_id	1
	US		2
	Free		3
Row 2	2	country	US
	UK		UK
	Paid		ES
Row 3	3	subscription_type	Free
	ES		Paid
	Paid		Paid

Errenkadaka (CSV, Avro): erregistro bakoitzaren eremuak batera gordetzen dira. **Erregistro osoak irakurtzeko egokia.**

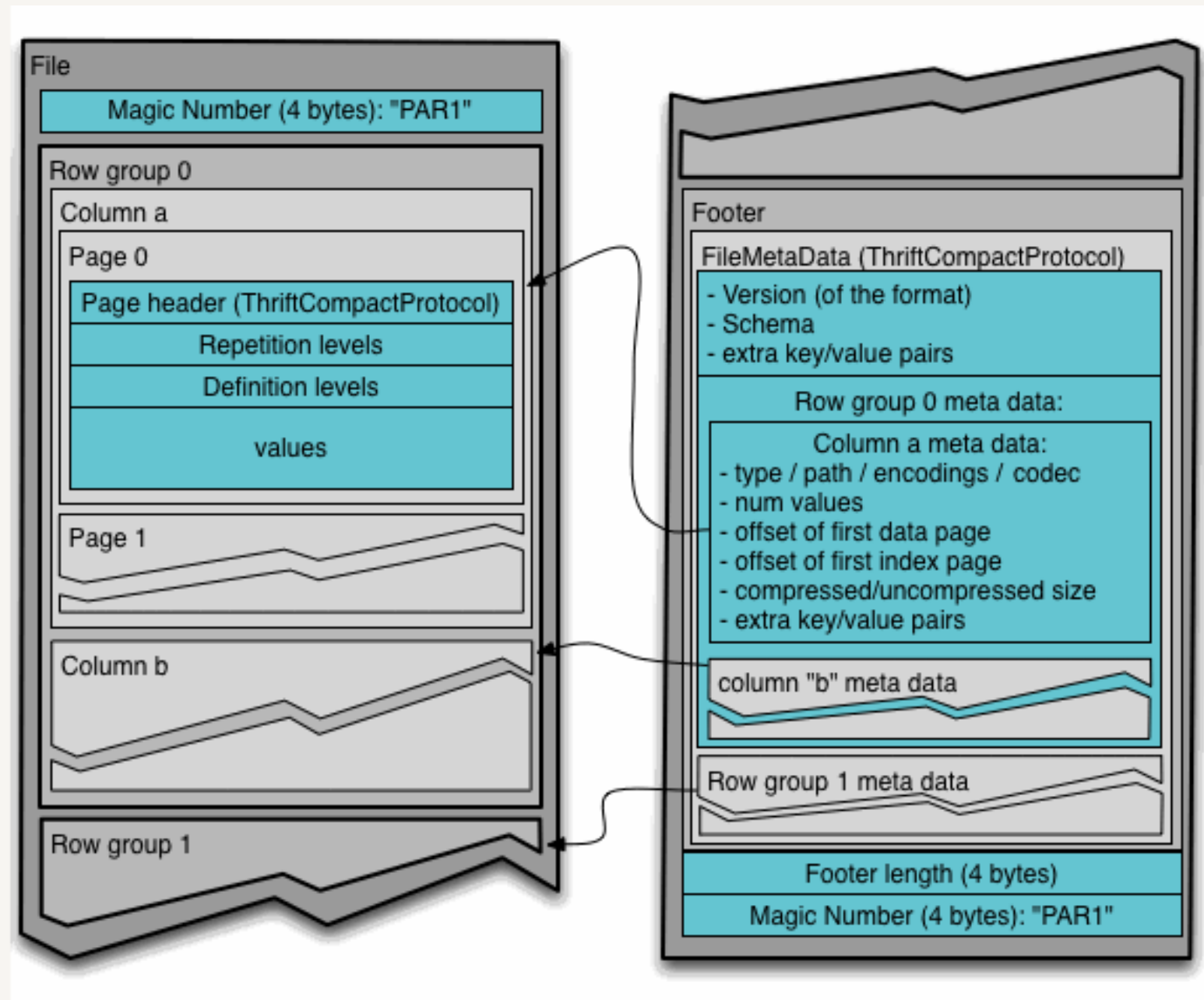
Zutabeka (Parquet, ORC): zutabe bereko balioak batera antolatzen dira. **Zutabe jakinak irakurtzean I/O murrizten du.**

JSON / JSONL (JSON Lines): dokumentuetan oinarritutako testu-formatuak. JSONL-n lerro bakoitza JSON objektu independentea da; datu-fluxuak eta prozesamendu banatua errazten ditu.



— APACHE PARQUET

Parquet: Zutabe-Formatua



- **Zutabeetan oinarritutako formato binarioa** – analitika egiteko optimizaturia
- **Auto-deskribatzailea** – eskema eta metadatuak fitxategian bertan
- **Row groups** – datuak bloketan antolatzen ditu, zutabeka
- **Beharrezko zutabeak soilik irakur daitezke** — I/O gutxiago
- **Konpresio eraginkorra** – Snappy, Gzip, Zstandard...
- **Spark, Athena BigQuery, Snowflake...** - oso erabilia analitika-ekosisteman
- **Metadatuak fitxategiaren amaieran gordetzen ditu**

06

Faker datu sintetikoak

Datu Sintetikoak · Definizioa eta Erabilera

Sintetikoak

Erabilera-kasu zehatz baterako **datu errealen ezaugarriak imitatzen dituzten datu artifizialak**. Garapen, proba eta ikasketa automatikorako erabil daitezke, datu errealak eskuragarri ez daudenean, nahikoak ez direnean edo erabiltzea komeni ez denean.

PRIBATUTASUNA

Datu sintetikoek **datu pertsonalen erabilera murrizten** lagun dezakete, benetako pertsonen erregistroak zuzenean erabili beharrean datu artifizialak erabiliz.

Zergatik Datu Sintetikoak? 5 Arrazoi

- **Pribatutasuna babestu** — datu pertsonal errealen erabilera murriztu
- **Sistema berriak garatu** — oraindik benetako daturik ez dagoenean probatu
- **ML ereduak entrenatu** — datu errealak urriak edo desorekatuak direnean osatu
- **Errendimendu-probak** — sistema karga handian probatu, milioika erregistrorekin
- **Segurtasun-probak** — probak egin benetako datu sentikorrek erabili gabe

Faker · Lehen Urratsak

```
●●● holaFaker.py python

1  # Instalatu
2  # pip3 install faker
3
4  from faker import Faker
5
6  fake = Faker('es_ES')    # Lokalizazioa
7
8  print(fake.name())
9  print(fake.address())
10 print(fake.phone_number())
11 print(fake.email())
12 print(fake.date())
13 print(fake.sentence(nb_words=10))
```

`FAKER('es_ES')` - Lokalizazioa

Izenak, helbideak, telefonoak... Espainiako formatuetara egokitzen ditu.

METODO EZBERDINAK

`name()`, `address()`, `phone_number()`, `email()`, `date()`... Metodo bakoitzak datu sintetiko mota bat sortzen du.

Faker 'es_ES' · Emaita Adibidea

●●● emailza.txt

output

```
1 Izena:          Dani Pla Chico
2 Helbidea:       Cuesta de Emiliano Milla 66, Albacete, 83227
3 Telefono-zenbakia:+34 818 779 827
4 Emaila:         btome@example.net
5 Data:          1984-09-29
6 Testu sintetikoa:Perferendis saepe consequatur saepe sapiente est impedit...
7
8 # Exekuzio bakoitzean datu desberdinak sortzen dira
9 # Emaita erreproduzigarriak lortzeko: Faker.seed(42) erabili
```

1000 Erregistroko CSV Sortzea

```
generaCSV.py python

1 from faker import Faker
2 import csv
3
4 fake = Faker('es_ES')
5 with open('datuakFaker.csv', 'w', newline='', encoding='utf-8') as irteera:
6     writer = csv.writer(irteera)
7
8     goiburua = ['izena', 'adina', 'hiria', 'longitudea', 'latitudea']
9     writer.writerow(goiburua)
10
11     for r in range(1000):
12         writer.writerow([fake.name(), fake.random_int(min=18, max=80),
13                          fake.city(), fake.longitude(), fake.latitude()])
```


JSON Egitura Antolatua Sortzea

```
•••  generaJSON.py  python
1  from faker import Faker
2  import json
3
4  fake = Faker('es_ES')
5  datuak = {'erregistroak': []}
6
7  for x in range(1000):
8      pertsona = {
9          "izena": fake.name(),
10         "adina": fake.random_int(18, 80),
11         "longitudea": float(fake.longitude()),
12     }
13     datuak['erregistroak'].append(pertsona)
14
15  with open('datuak.json', 'w', encoding='utf-8') as f:    json.dump(datuak, f, indent=2, ensure_ascii=False)
```

Faker Provider-ak · Kategoria Nagusiak 1/2

PROVIDER	METODO ADIBIDEAK
person	name() · first_name() · last_name() · prefix()
address	address() · city() · state() · postcode() · country()
internet	email() · url() · domain_name() · ipv4() · user_agent()
phone_number	phone_number() · msisdn()
date_time	date() · time() · date_time() · date_of_birth()
company	company() · company_suffix() · job()

Faker Provider-ak · Kategoria Nagusiak 2/2

PROVIDER	METODO ADIBIDEAK
lorem	text() · sentence() · paragraph() · words()
bank	iban() · bban() · swift()
geo	latitude() · longitude() · local_latlng()
color	color_name() · hex_color() · rgb_color()
misc	boolean() · uuid4() · md5() · sha256()
barcode	ean() · ean13() · ean8()

ESKERRIK ASKO

Galderak, iruzkinak.